

Politecnico di Torino

POLITECNICO DI TORINO

Corso di laurea in
Ingegneria Matematica

Computational linear algebra for large scale problems - Homework 2 - Spectral Clustering

Lucia Ghezzi s325951
Giuliana Paradiso s319688

15 Gennaio 2025

Contents

1 Introduction **4**

2 K-nearest neighborhood similarity graph and its adjacency matrix W **4**
2.1 Similarity matrix S 4
2.2 Adjacency matrix W 5
2.3 Similarity graph 5
2.3.1 Circle data-set 5
2.3.2 Spiral data-set 6

3 Degree matrix and Laplacian matrix **7**
3.1 Degree matrix D 7
3.2 Laplacian matrix L 7

4 Connected components of the similarity graph **8**

5 Suitable number of clusters **8**
5.1 Circle dataset 9
5.2 Spiral dataset 10

6 Computation of eigenvectors corresponding to smallest eigenvalues **11**

7 Spectral clustering performance **11**

8 Assignment of the initial points to clusters **11**

9 Clusters plots (M=3) **12**
9.1 Circle data-set 12
9.1.1 k=10 12
9.1.2 k=20 13
9.1.3 k=40 13
9.1.4 Observations 14
9.2 Spiral dataset 15
9.2.1 k=10 15
9.2.2 k=20 16
9.2.3 k=40 17
9.2.4 Observations 17

10 Comparison with other clustering methods **18**
10.1 Circle dataset 18
10.1.1 k=10 18
10.1.2 k=20 19
10.1.3 k=40 20
10.1.4 Observations 20
10.2 Spiral dataset 21

10.2.1 k=10 . . . 21
10.2.2 k=20 . . . 22
10.2.3 k=40 . . . 23
10.2.4 Observations . . . 23

11 Results for different numbers of clusters **24**

12 Self implemented functions **26**
12.1 *inverse_power_method* . . . 26
12.2 *deflation_method* . . . 26
12.3 *compute_m_smallest_eigenvalues* . . . 27

13 Conclusions **28**

List of Figures

1 Similarity graph for the Circle data-set . . . 6
2 Similarity graph for the Spiral dataset . . . 7
3 Plot of the smallest eigenvalues of L for the Circle dataset . . . 9
4 Plot of the smallest eigenvalues of L for the Spiral dataset . . . 10
5 Comparison between the clusters obtained with spectral clustering, using the Laplacian matrix or its symmetrized version, k=10 . . . 12
6 Comparison between the clusters obtained with spectral clustering, using the Laplacian matrix or its symmetrized version, k=20 . . . 13
7 Comparison between the clusters obtained with spectral clustering, using the Laplacian matrix or its symmetrized version, k=40 . . . 13
8 Comparison between the clusters obtained with spectral clustering, using the Laplacian matrix or its symmetrized version, and the clustering obtained with the original labels, k=10 . . . 15
9 Comparison between the clusters obtained with spectral clustering, using the Laplacian matrix or its symmetrized version, k=20 . . . 16
10 Comparison between the clusters obtained with spectral clustering, using the Laplacian matrix or its symmetrized version, k=40 . . . 17
11 Clusters of points for the Circle datasets obtained with other clustering methods, k=10 . . . 18
12 Clusters of points for the Circle datasets obtained with other clustering methods, k=20 . . . 19
13 Clusters of points for the Circle datasets obtained with other clustering methods, k=40 . . . 20
14 Clusters of points for the Spiral data-sets obtained with other clustering methods, k=10 . . . 21
15 Clusters of points for the Spiral data-sets obtained with other clustering methods, k=20 . . . 22
16 Clusters of points for the Spiral data-sets obtained with other clustering methods, k=40 . . . 23

17 Clusters of points for the Circle and Spiral data-sets obtained with other spectral clustering, k=10, M=5 24
18 Clusters of points for the Circle and Spiral data-sets obtained with other spectral clustering, k=10, M=9 25
19 Clusters of points for the Circle and Spiral data-sets obtained with other spectral clustering, k=10, M=12 25

1 Introduction

This report aims to describe how spectral clustering was carried out for two datasets of points, Circle.mat, containing two columns corresponding to x-values and y-values, and Spiral.mat, containing three columns, the third one containing the index of the correct cluster.
The first part of the report (sections from 1 to 8), presents an explanation of how the various steps of the spectral clustering were implemented. In particular, it contains a description of how the fundamental objects needed to implement spectral clustering, such as the adjacency, degree and Laplacian matrices, were obtained, and a description of how the most suitable number of clusters was chosen.
The second part (section 9), presents the results obtained by applying spectral clustering to the desired datasets, and how these results change with the parameter k, used for the construction of the adjacency matrix.
The third part (section 10 and 11), presents comparisons between the clusters of the Circle and Spiral datasets obtained with the self-implemented spectral clustering and the clusters for the same datasets obtained with other clustering methods, such as the k-means and k-medoids algorithms, and the Matlab spectral clustering algorithm. In addiction, it contains the cluster results for the datasets obtained using different numbers of clusters.
Finally, the last part of this report (section 12) focuses on the self-implemented functions that make possible the computation of the m smallest eigenvalues by applying the Inverse Power method combined with deflation. All the methods have been implemented using Matlab software.

2 K-nearest neighborhood similarity graph and its adjacency matrix W

2.1 Similarity matrix S

Given a similarity function s_{ij} for two points, the similarity graph is denoted by $G = (V, E)$ where $V = v_1, ..., v_n$ denotes a non-empty set of vertices and E denotes the set of edges, i.e., a set of pair of vertices. In the similarity graph each vertex $v_i \in V$ represents a data point X_i. An edge between two vertices v_i and v_j exists if the similarity s_{ij} between the corresponding data points X_i and X_j is either positive or larger than a certain threshold (minumum similarity value ϵ for a connection to take place between two data points). We assume that $s_{ij} = s_{ji}$ and that the edge connecting v_i and v_j is weighted by s_{ij}. Consequently, it turns out that the similarity graph is undirected.
The similarity function is

$$s_{ij} = exp\left(\frac{-||X_i - X_j||^2}{2\sigma^2}\right)$$

where $\sigma = 1$.
The similarity matrix $S = s_{ij}(X_i, X_j)$ for $i, j \in J$ is obtained by computing the upper triangular part of the matrix S and extending it to the full matrix to make it symmetric. We notice that the diagonal of the matrix is null since $||X_i - X_i|| = 0$.
To obtain the k-nearest neighborhood similarity graph we compute the k highest values of

each row of S and their respective indices, respectively saved in the matrices K and I, via the Matlab function maxk.

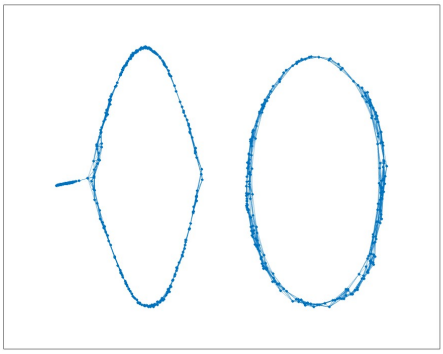
2.2 Adjacency matrix W

The weighted adjacency matrix is defined as $W_{ij} = s_{ij}$ if $i \neq j$ and $W_{ij} = 0$ if $i = j$.
It is obtained by first saving the k highest values of each row of S in the matrix, and then symmetrizing the matrix itself. The symmetrization was carried out in this way: we consider two nodes i and j to be k-neighbors if j is one of the k-neighbors of j or j is one of the k-neighbors of i.

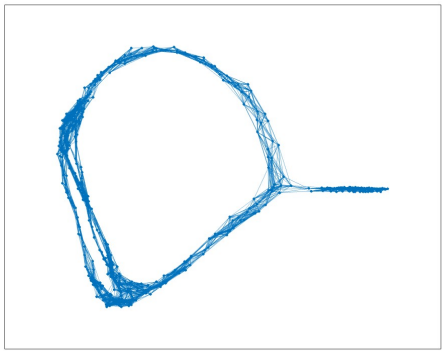
2.3 Similarity graph

The similarity graph G is then obtained creating a graph object, with the adjacency matrix W.

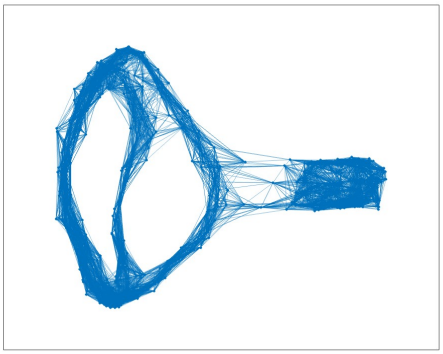
2.3.1 Circle data-set



(a) k=10



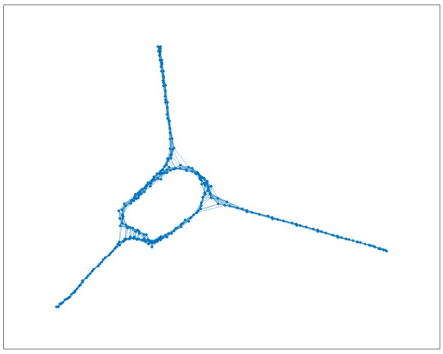
(b) k=20



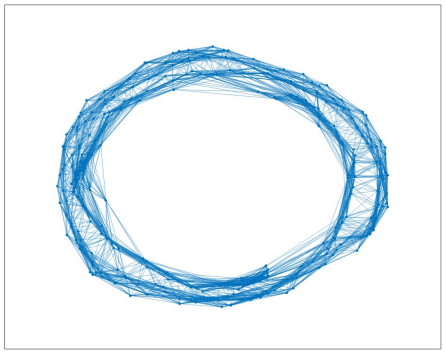
(c) k=40

Figure 1: Similarity graph for the Circle data-set

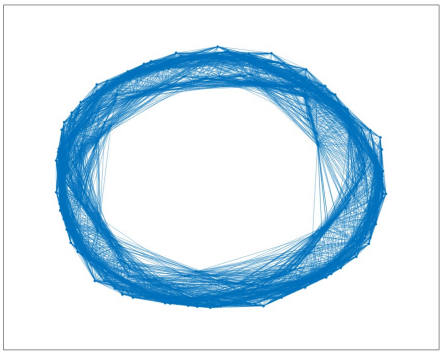
2.3.2 Spiral data-set



(a) k=10



(b) k=20



(c) k=40

Figure 2: Similarity graph for the Spiral dataset

3 Degree matrix and Laplacian matrix

3.1 Degree matrix D

The degree matrix D is a diagonal matrix having as diagonal the vector d st $d_i = \sum_{j=1}^{n} W_{i,j}$. The elements of d are computed when constructing W and D is obtained with the Matlab function diag(d).
The Degree matrix is saved in the sparse format via the Matlab function sparse.

3.2 Laplacian matrix L

The Laplacian matrix L is defined as

$$L = D - W$$

It is computed by simply computing the difference of the two sparse matrices D and W, which is still a sparse matrix.

The normalized symmetric Laplacian matrix L_{sym} is defined as $L_{sym} = I - D^{-1/2}WD^{-1/2}$ and it is computed by matrix multiplication and $D^{-1} = diag(1/d)$.

4 Connected components of the similarity graph

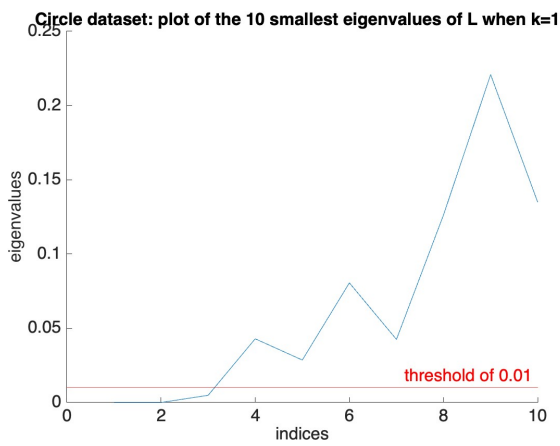
The number of connected components of the similarity graph is obtained by computing the maximum value of the Matlab function conncomp(G).
The results show (as it is confirmed by the graphs representations of Section 2.3) that for the Circle dataset, the 10-nearest neighborhood similarity graph has two connected components, while both the 20-nearest and 40-nearest neighborhood similarity graphs have just one connected component. For the Spiral data-set, on the other hand, all three similarity graphs have one connected component.

5 Suitable number of clusters

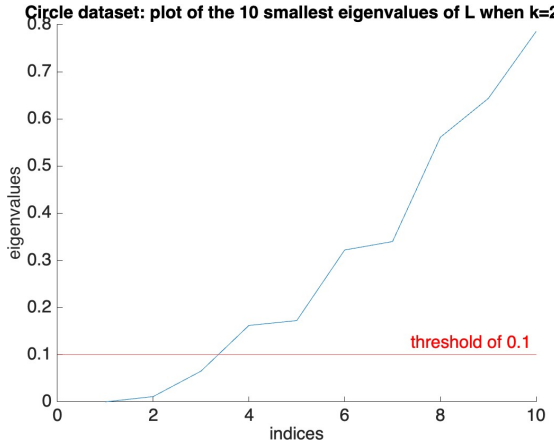
To choose a suitable number of clusters M for the datasets, the self-implemented function *compute_m_smallest_eigenvalues* was exploited (an accurate description of the function and the mathematical theory behind it can be found in Section 12.3.
To explain the choice of M, let us recall that the number of null eigenvalues is equal to the number of connected components of the graph, and non-null eigenvalues represent the strength of the connectivity between the connected components of the graph. Therefore, for both datasets the plot of the smallest m> M eigenvalues was useful to choose the appropriate number of clusters.

5.1 Circle dataset

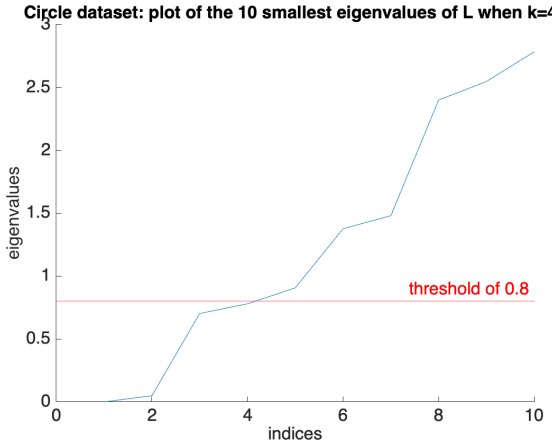
In the three cases, for k=10, k=20 and k=40, there are 3 eigenvalues below a very small threshold, which suggests taking as number of clusters M=3. Nonetheless, since there are more than 3 small eigenvalues, it would also be interesting to try to apply spectral clustering imposing a higher number of clusters (see section 11).



(a) k=10



(b) k=20

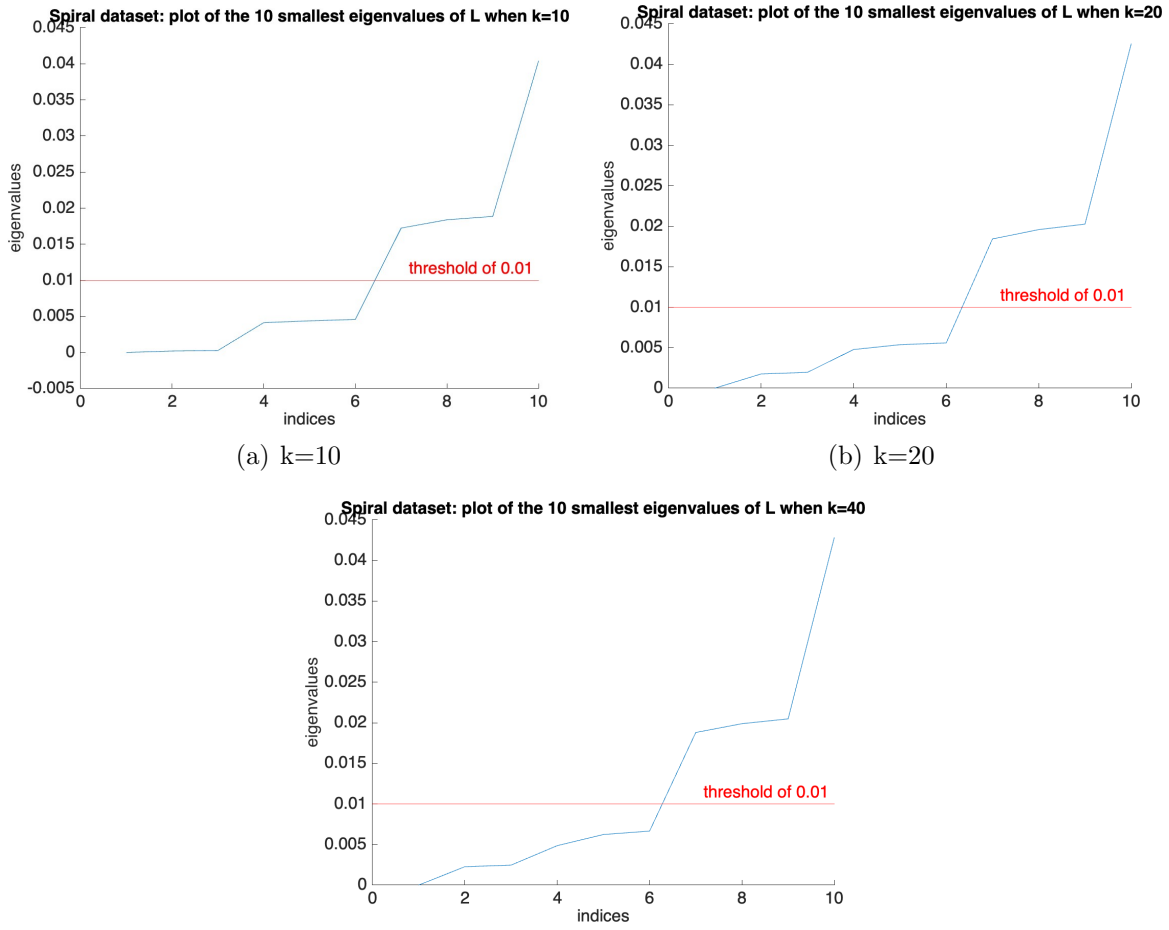


(c) k=40

Figure 3: Plot of the smallest eigenvalues of L for the Circle dataset

5.2 Spiral dataset

Similar considerations also hold for the Spiral dataset: in the three cases k=10, k=20, k=40, there are 3 eigenvalues that are significantly smaller than the others, which suggests taking M=3 clusters; nonetheless, being the number of very small eigenvalues greater than 3, it would also be interesting to apply spectral clustering with M $>$ 3.



(a) k=10

(b) k=20

(c) k=40

Figure 4: Plot of the smallest eigenvalues of L for the Spiral dataset

6 Computation of eigenvectors corresponding to smallest eigenvalues

The computation of the M=3 smallest eigenvalues and corresponding eigenvectors of the Laplacian matrix for the Circle and Spiral datasets is obtained with the self-implemented function *compute_m_smallest_eigenvalues*. The function returns a vector containing the smallest eigenvalues and a matrix U having as columns the corresponding m eigenvectors. A detailed explanation of how the function works is provided in Section 12.3.

7 Spectral clustering performance

Spectral clustering is performed by applying the kmeans algorithm to the n rows of the matrix U and storing, for every point, the index of its assigned cluster in the *clusters* vector. This is simply done by using the Matlab function kmeans(U, M).

8 Assignment of the initial points to clusters

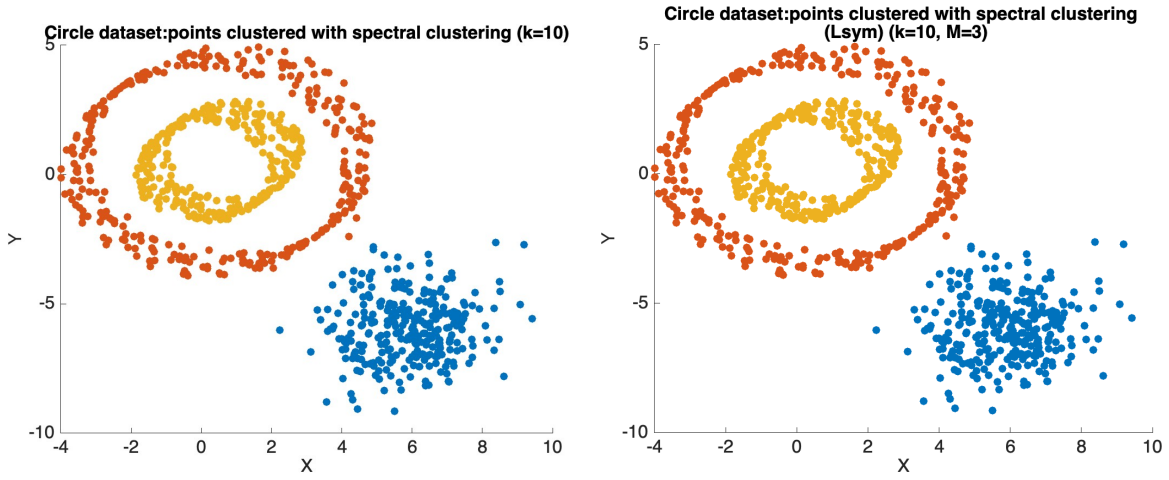
Each point of the original dataset is assigned to the same cluster to which the corresponding row in U is assigned by kmeans. Clusters $C_1, C_2, ..., C_M$ are stored in a dictionary datatype, where keys are the cluster names (numbers) and values are the points assigned to each cluster.

9 Clusters plots (M=3)

This section reports the plots of the M=3 clusters of points of the two datasets, as the value of the parameter k changes.

9.1 Circle data-set

9.1.1 k=10

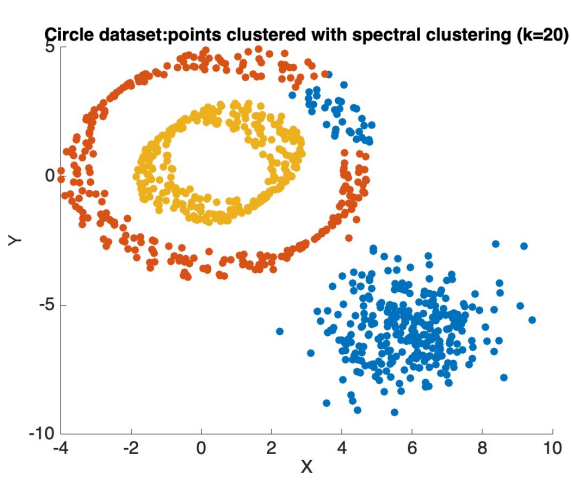


(a) Clusters of points for the Circle dataset

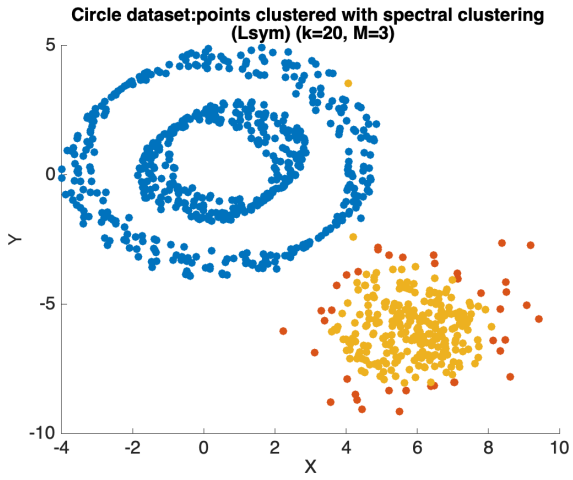
(b) Clusters of points for the Circle data-set obtained using Lsym

Figure 5: Comparison between the clusters obtained with spectral clustering, using the Laplacian matrix or its symmetrized version, k=10

9.1.2 k=20



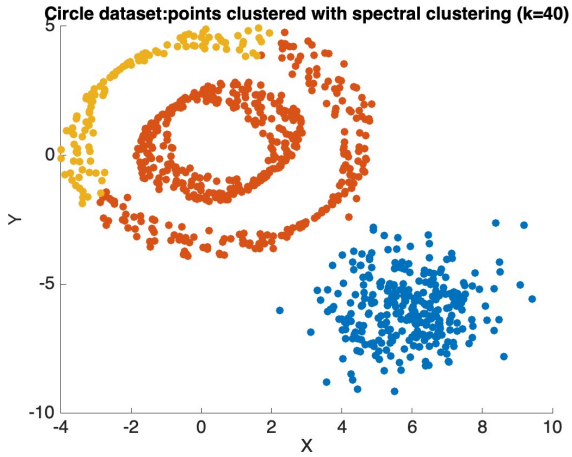
(a) Clusters of points for the Circle data-set



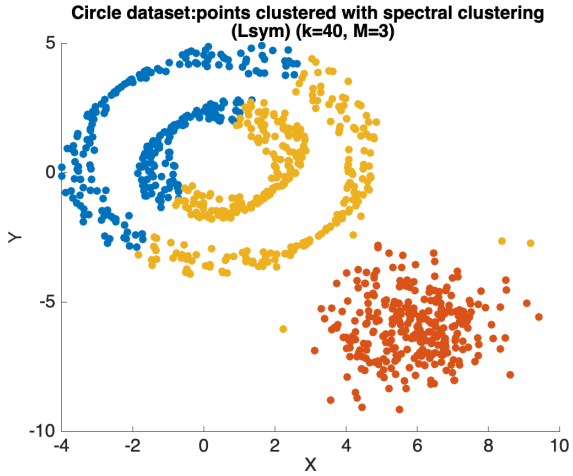
(b) Clusters of points for the Circle data-set obtained using Lsym

Figure 6: Comparison between the clusters obtained with spectral clustering, using the Laplacian matrix or its symmetrized version, k=20

9.1.3 k=40



(a) Clusters of points for the Circle data-set



(b) Clusters of points for the Circle data-set obtained using Lsym

Figure 7: Comparison between the clusters obtained with spectral clustering, using the Laplacian matrix or its symmetrized version, k=40

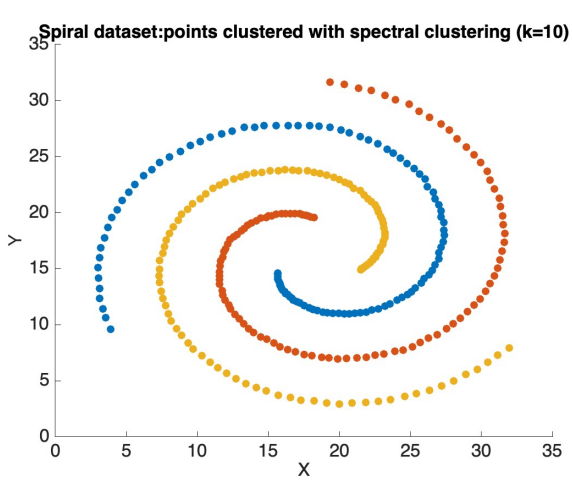
9.1.4 Observations

As it can be seen from the plots of the clusters of points of the Circle dataset, the ideal number of k-neighbors to construct the adjacency matrix W (and consequently the Laplacian matrix L) is around $k = 10$. For higher values of k, in fact, clusters become less and less accurate. It is still possible to notice that the $k = 40$ setting is preferable to the $k = 20$ setting.
Moreover, for $k = 10$ there is no relevant difference in the results of the spectral clustering performance when using L or its normalized version L_{sym}; differently, for $k = 20$, using L_{sym} provides a clustering result that separates outliers but does not distinguish between the two circles; finally, for $k = 40$, the performance of spectral clustering with the normalized L_{sym} is similar to the one we would expect from applying a centroid-based algorithm directly on the initial dataset.

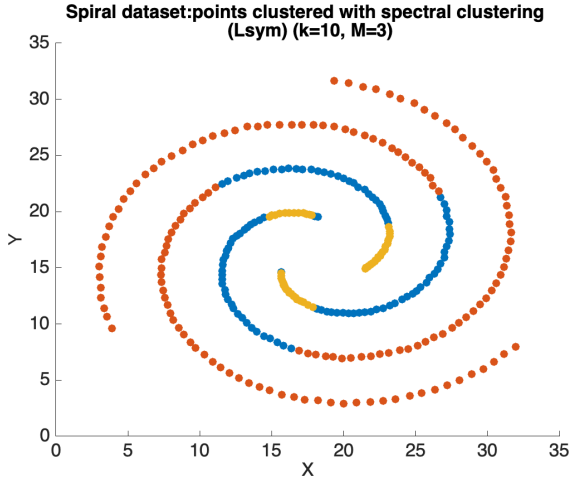
9.2 Spiral dataset

Since the correct cluster indices for the Spiral dataset are available within the dataset itself, we can showcase the comparison between the correct clusters and the ones obtained with the spectral clustering method.

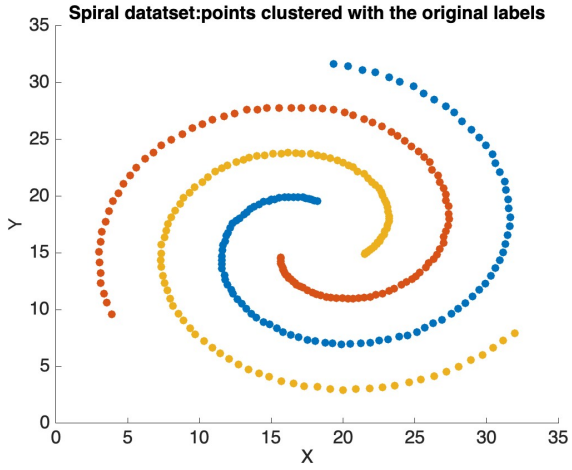
9.2.1 k=10



(a) Clusters of points for the Spiral dataset



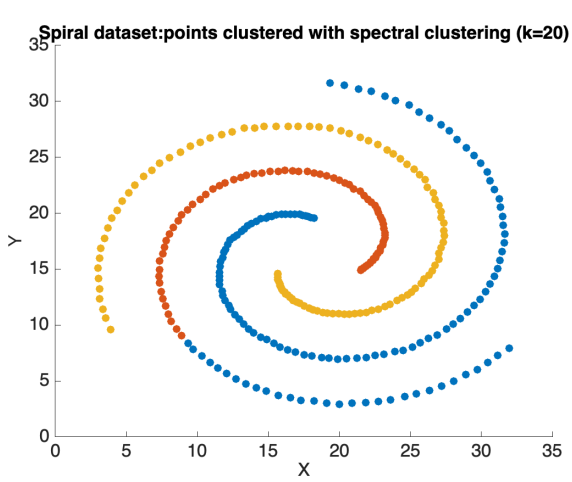
(b) Clusters of points for the Spiral dataset using Lsym



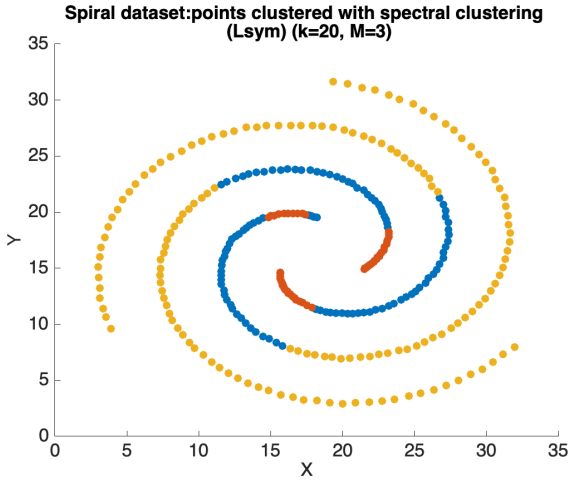
(c) Clustering with original labels

Figure 8: Comparison between the clusters obtained with spectral clustering, using the Laplacian matrix or its symmetrized version, and the clustering obtained with the original labels, k=10

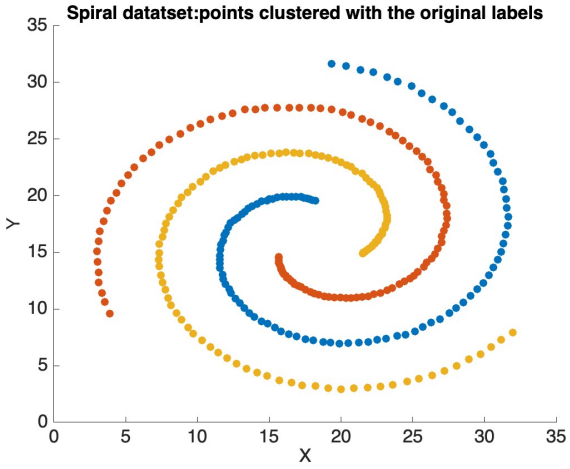
9.2.2 k=20



(a) Clusters of points for the Spiral data-set



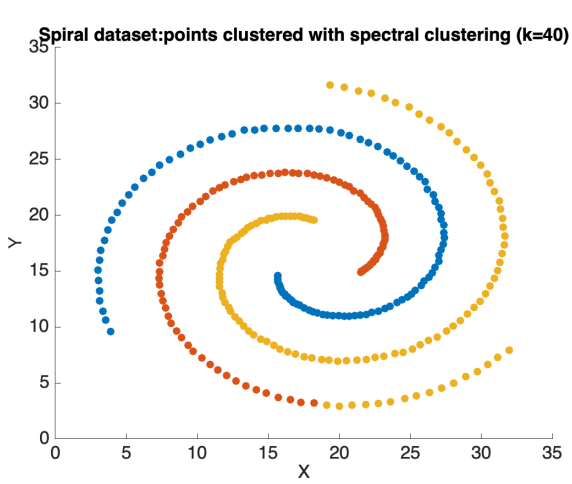
(b) Clusters of points for the Spiral data-set using Lsymg



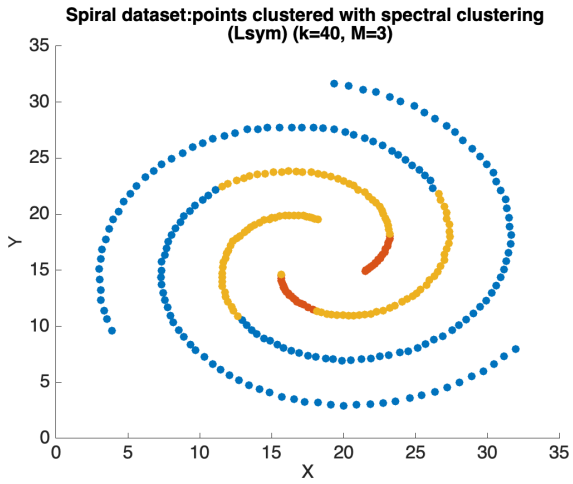
(c) Clustering with original labels

Figure 9: Comparison between the clusters obtained with spectral clustering, using the Laplacian matrix or its symmetrized version, k=20

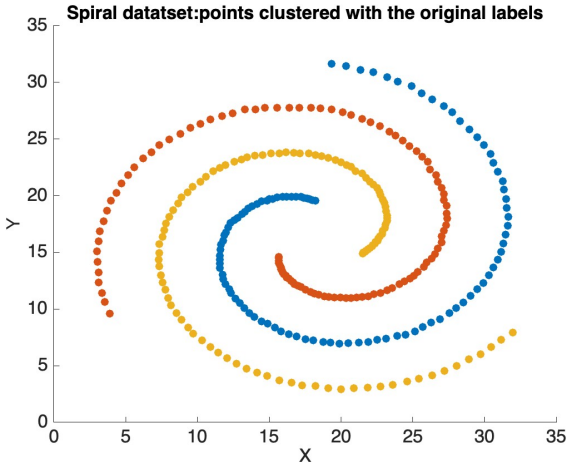
9.2.3 k=40



(a) Clusters of points for the Spiral data-set



(b) Clusters of points for the Spiral data-set using Lsymg



(c) Clustering with original labels

Figure 10: Comparison between the clusters obtained with spectral clustering, using the Laplacian matrix or its symmetrized version, k=40

9.2.4 Observations

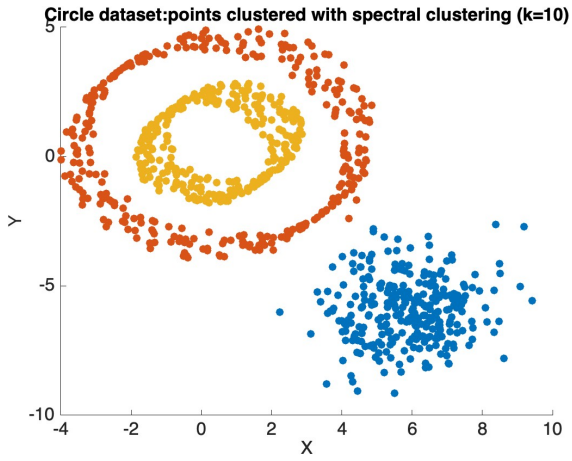
The previous comparisons show that the performance of the spectral clustering method gets worse as the number of k-neighbors of the adjacency matrix W (and consequently the Laplacian matrix L) increases: clusters are correct for $k = 10$ and begin to differ from the original clusters for $k = 20$ and $k = 40$.

10 Comparison with other clustering methods

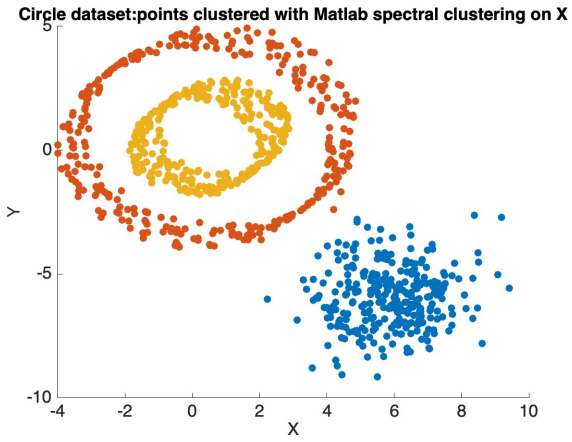
To assess the validity of the spectral clustering method implemented, comparison with other clustering methods is provided. The chosen clustering methods are the following: Matlab spectral clustering, k-means clustering, and k-medoids clustering.

10.1 Circle dataset

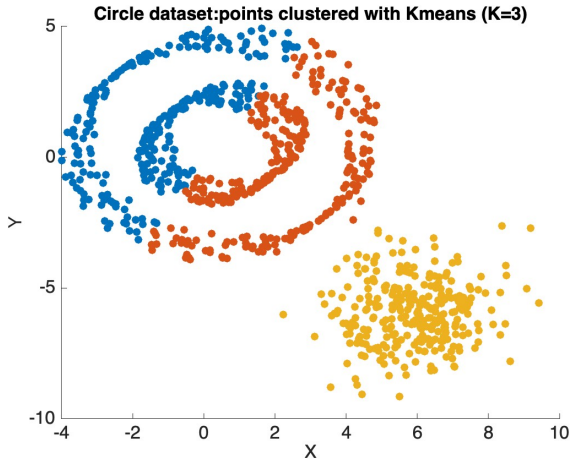
10.1.1 k=10



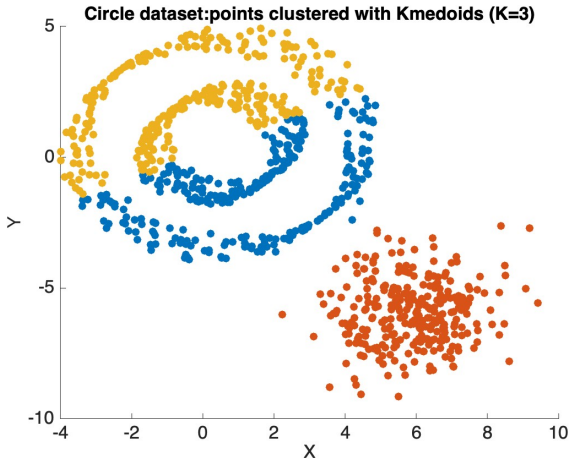
(a) Spectral clustering



(b) Matlab spectral clustering



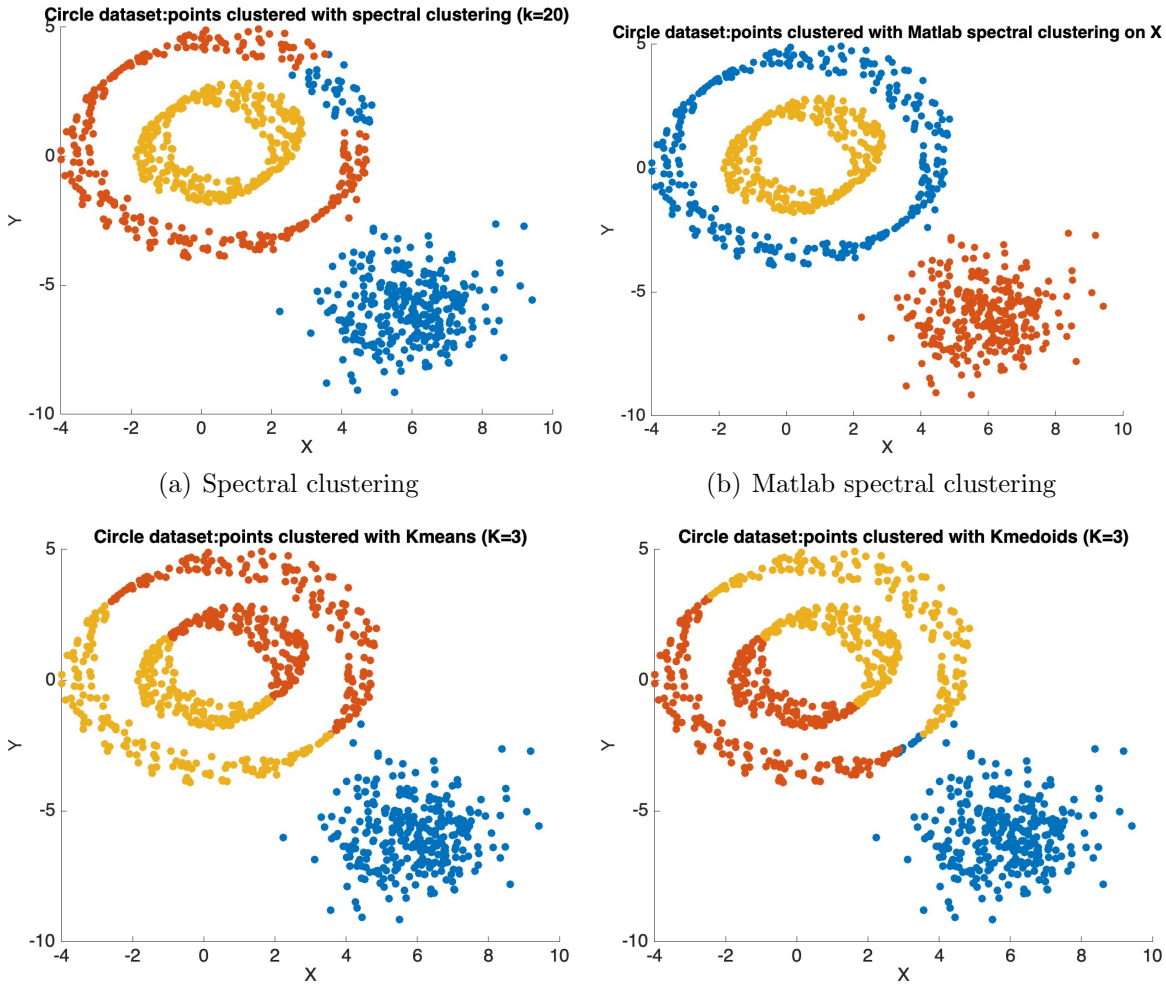
(c) K-means clustering



(d) K-medoids clustering

Figure 11: Clusters of points for the Circle datasets obtained with other clustering methods, k=10

10.1.2 k=20



(a) Spectral clustering

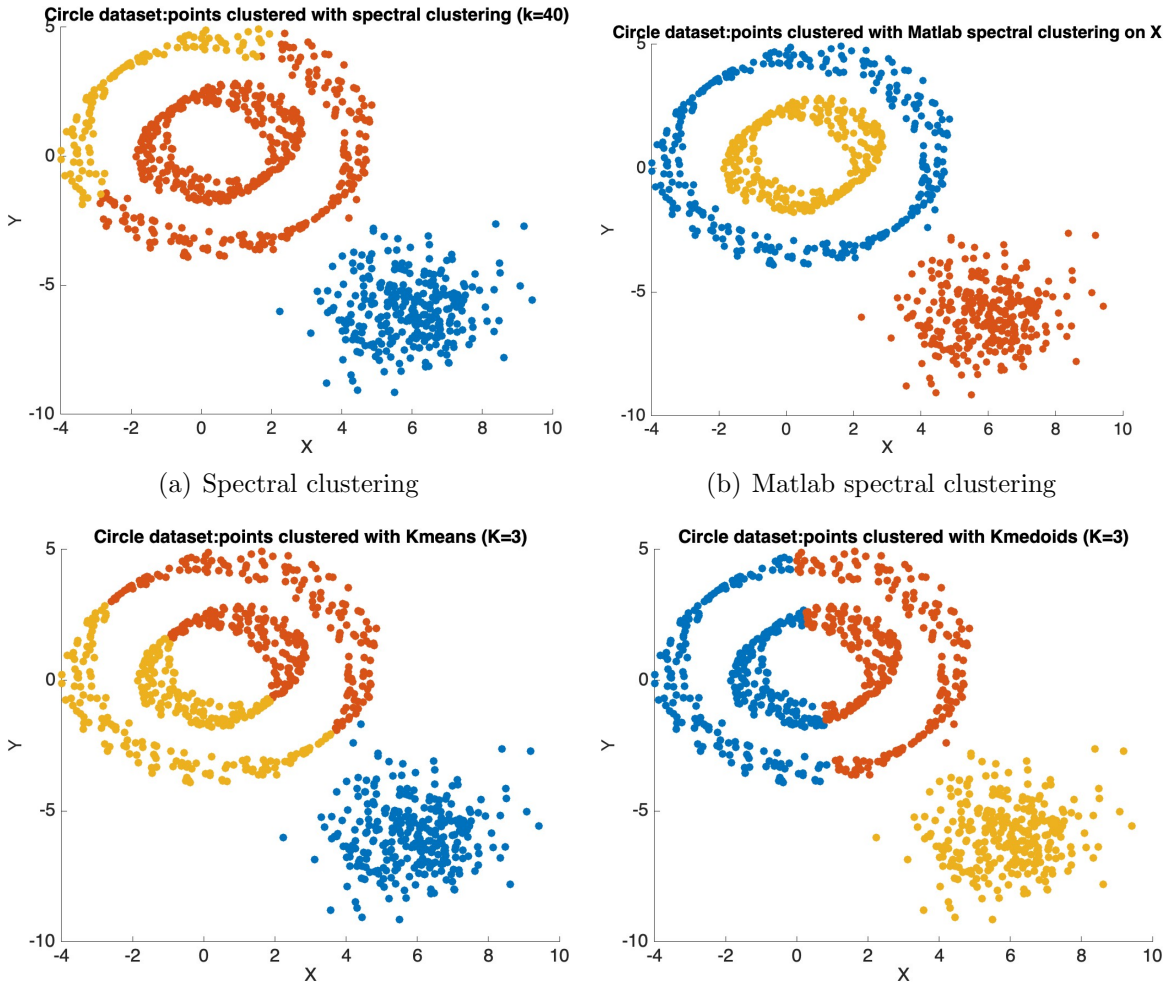
(b) Matlab spectral clustering

(c) K-means clustering

(d) K-medoids clustering

Figure 12: Clusters of points for the Circle datasets obtained with other clustering methods, k=20

10.1.3 k=40



(a) Spectral clustering

(b) Matlab spectral clustering

(c) K-means clustering

(d) K-medoids clustering

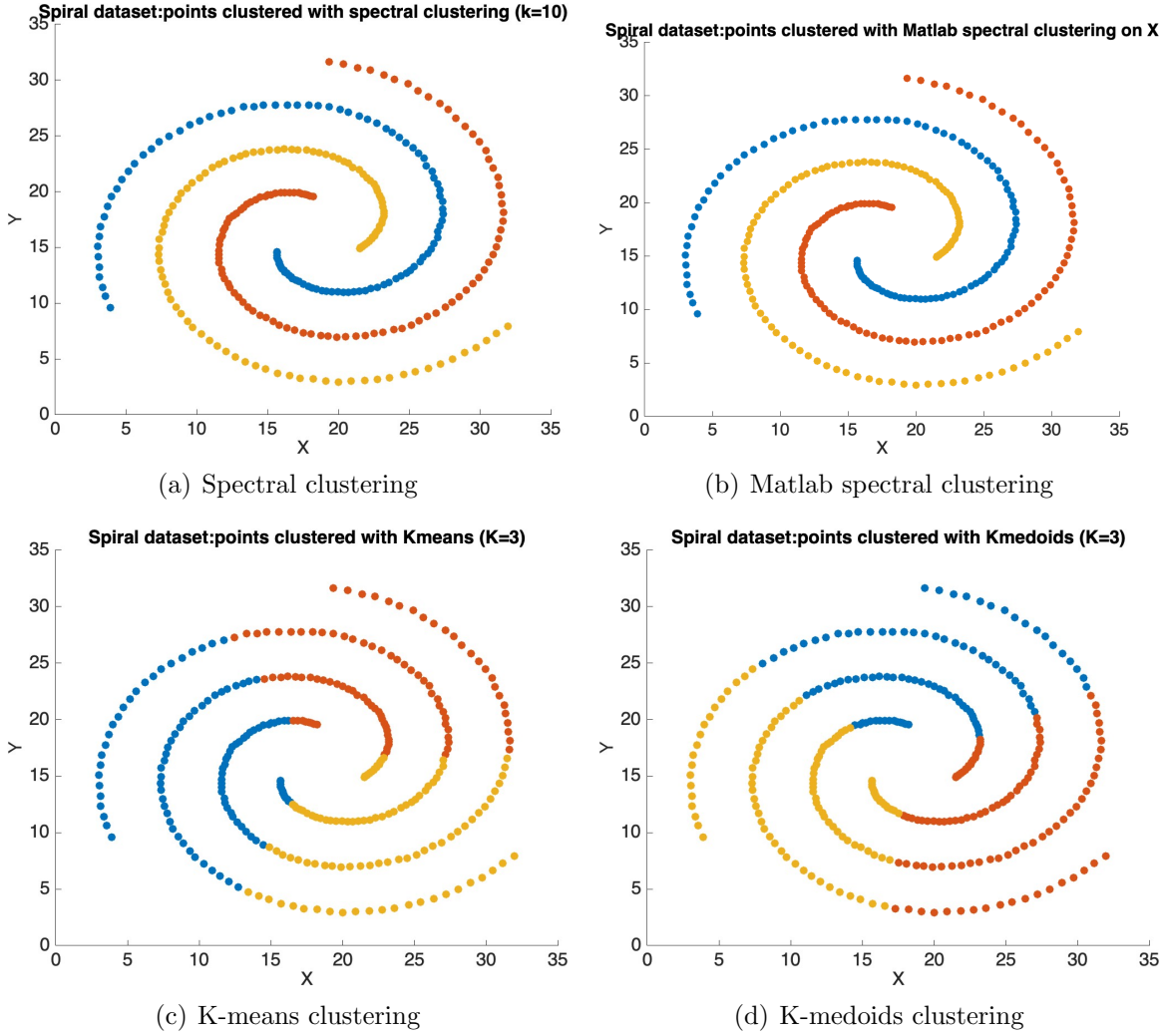
Figure 13: Clusters of points for the Circle datasets obtained with other clustering methods, k=40

10.1.4 Observations

In general, it is possible to notice that the clusters of points of the Circle dataset, obtained using the self-implemented spectral clustering method, tend to be more accurate than those generated by the k-means or k-medoids clustering methods, regardless of the chosen value of k. The centroid-based methods are, in fact, not suitable for the dataset structure itself, since data is not easy to divide using centroids.
Moreover, the difference between the clusters produced by the self-implemented spectral clustering method and those obtained with the Matlab method (which are equally valid for $k = 10$) becomes more pronounced as k increases.

10.2 Spiral dataset

10.2.1 k=10



(a) Spectral clustering

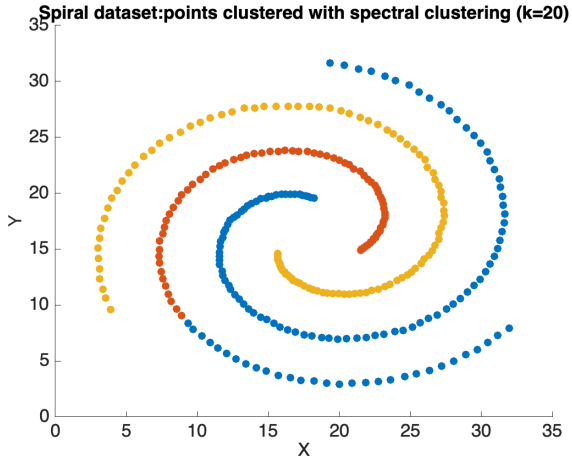
(b) Matlab spectral clustering

(c) K-means clustering

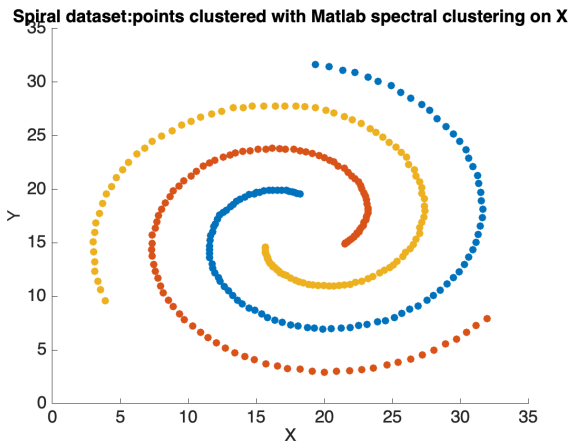
(d) K-medoids clustering

Figure 14: Clusters of points for the Spiral data-sets obtained with other clustering methods, k=10

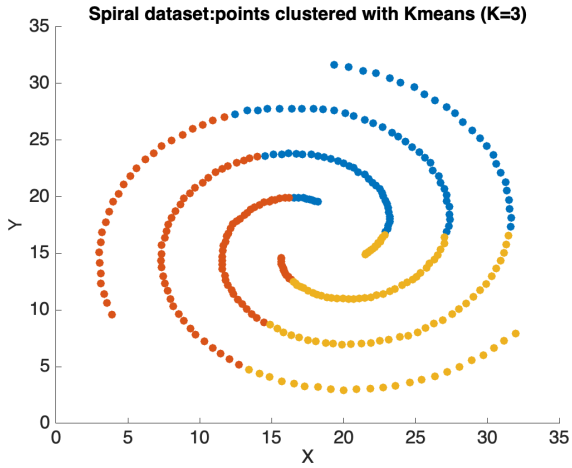
10.2.2 k=20



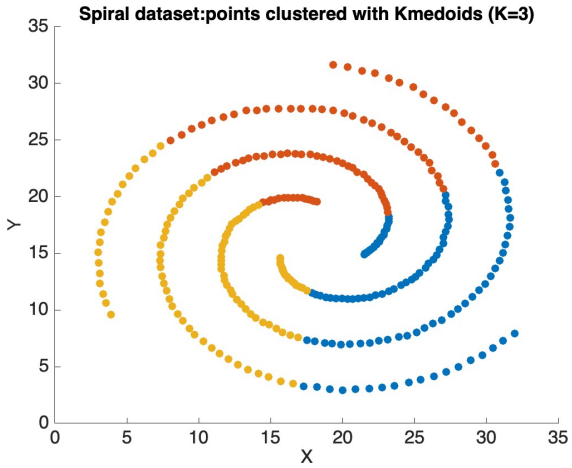
(a) Spectral clustering



(b) Matlab spectral clustering



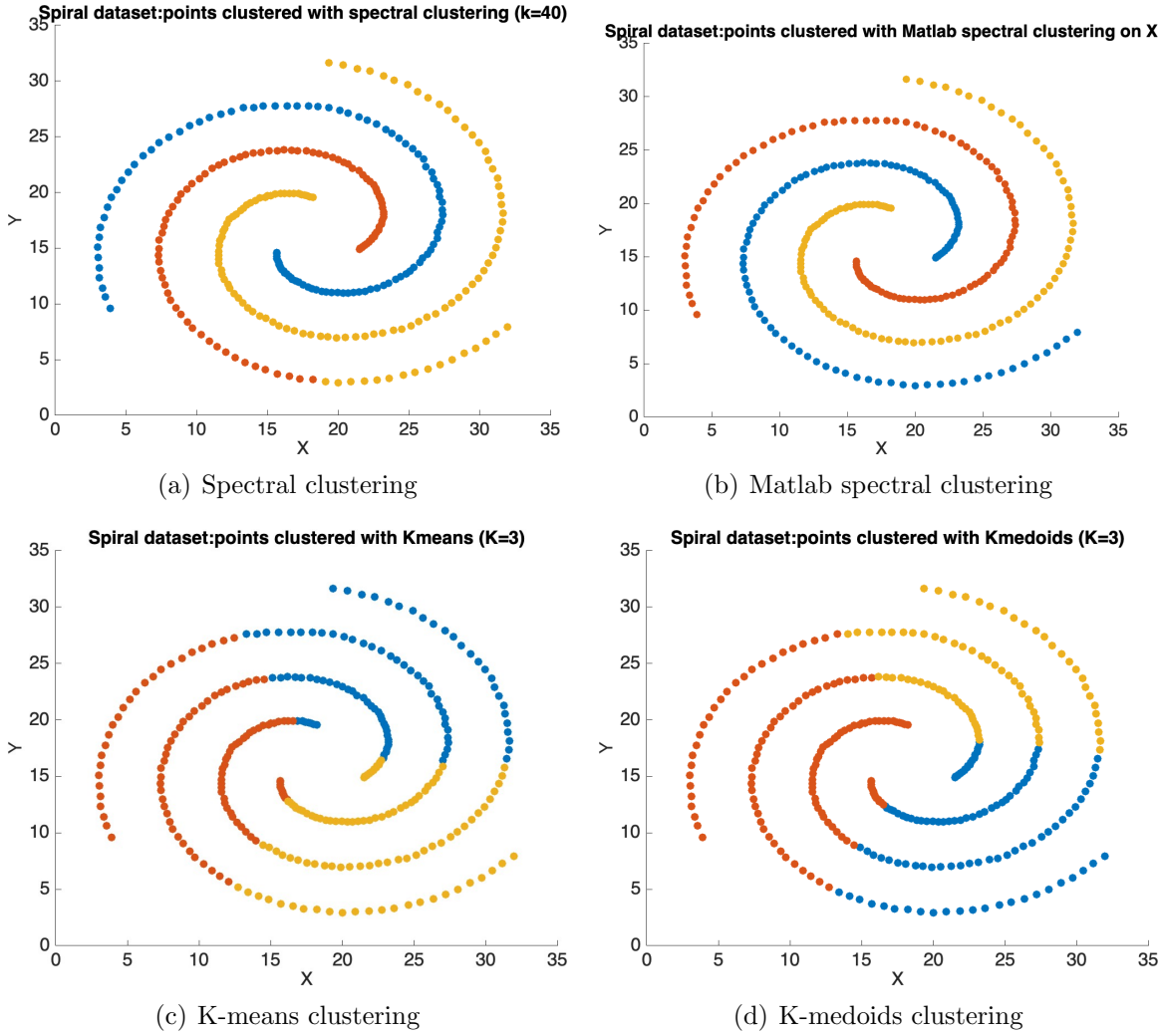
(c) K-means clustering



(d) K-medoids clustering

Figure 15: Clusters of points for the Spiral data-sets obtained with other clustering methods, k=20

10.2.3 k=40



(a) Spectral clustering

(b) Matlab spectral clustering

(c) K-means clustering

(d) K-medoids clustering

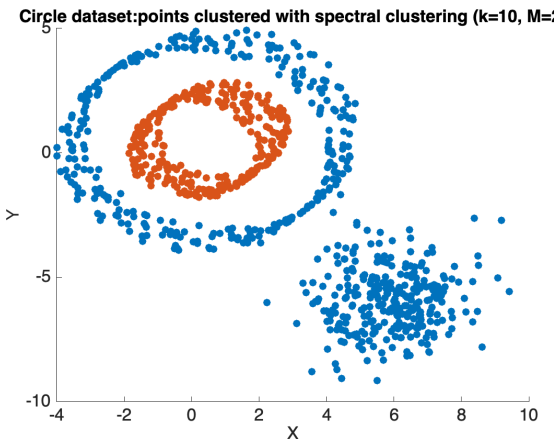
Figure 16: Clusters of points for the Spiral data-sets obtained with other clustering methods, k=40

10.2.4 Observations

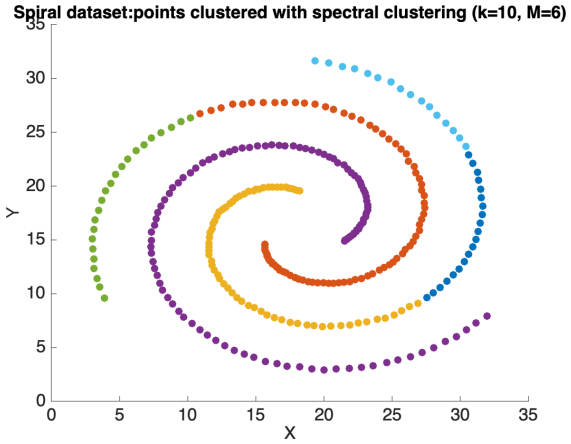
Similar considerations as for the Circle dataset hold also for the Spiral dataset. The manually-developed spectral clustering algorithm is equivalent to the Matlab one for small values of k and gets worse as k increases.
Nonetheless, its results are more aligned with the correct ones present within the dataset with respect to those obtained with k-means and k-medoids methods.

11 Results for different numbers of clusters

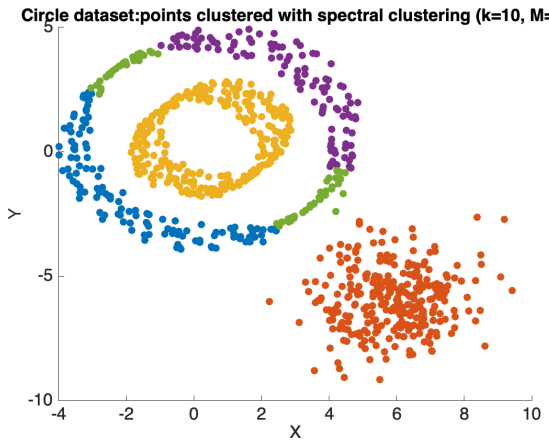
For both datasets, for $k = 10$, the number of close to zero eigenvalues is much greater than 3, so it may be interesting to try to produce more than 3 clusters. This section presents the clustered points of the Circle and Spiral datasets, obtained with the implemented spectral clustering method with parameter $k = 10$ and a range of different numbers of clusters M. As we could imagine, the clustering becomes less accurate as we decrease or increase the number of clusters. Three is, indeed, the optimal number of clusters for these datasets.



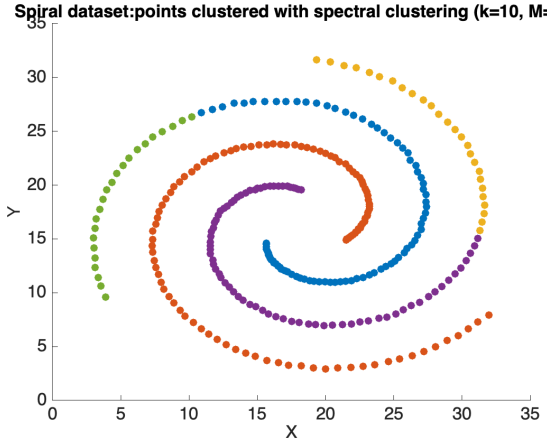
(a) Clusters of points for the Circle dataset, k=10, M=2



(b) Clusters of points for the Spiral dataset, k=10, M=6

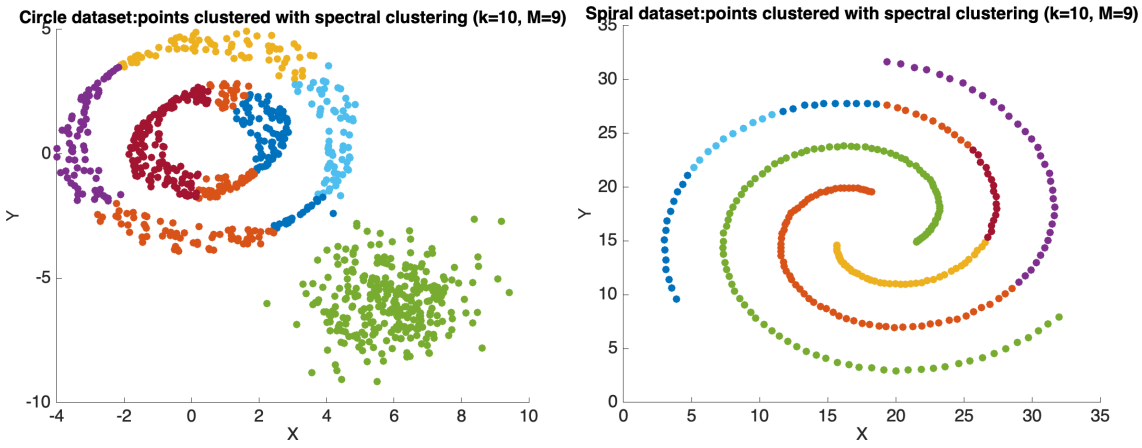


(c) Circle dataset



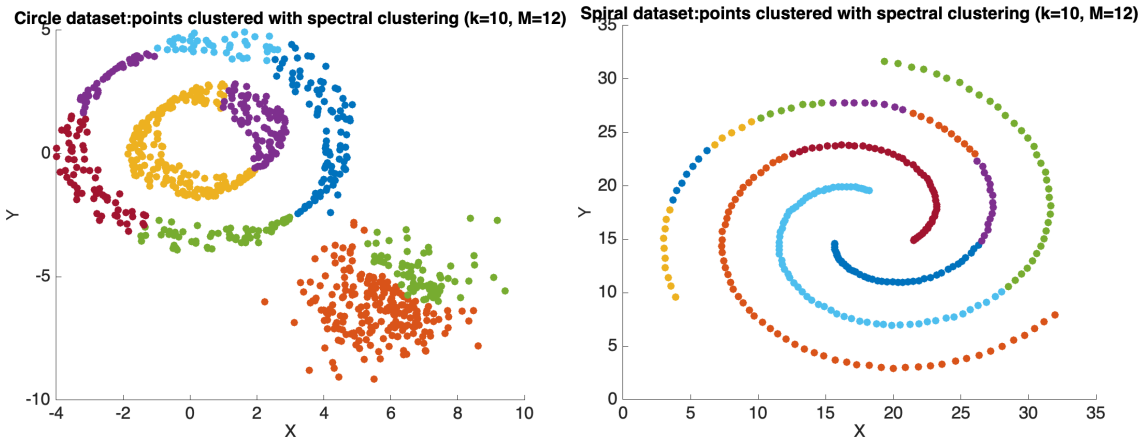
(d) Spiral dataset

Figure 17: Clusters of points for the Circle and Spiral data-sets obtained with other spectral clustering, k=10, M=5



(a) (b)

Figure 18: Clusters of points for the Circle and Spiral data-sets obtained with other spectral clustering, k=10, M=9



(a) (b)

Figure 19: Clusters of points for the Circle and Spiral data-sets obtained with other spectral clustering, k=10, M=12

12 Self implemented functions

The self implemented functions that were exploited to obtain the previously reported results are explained in the present section. Three Matlab functions were created: the first two to implement the *inverse power method* and the *deflation method* for eigenvalues, while the last one to compute a user-specified number m of smallest eigenvalues and corresponding eigenvectors of a given matrix A, combining the two methods.

12.1 *inverse_power_method*

The *inverse power method* exploits the fact that, for a non singular matrix A $\in \mathbf{R^{nxn}}$ having eigenvalues $|\lambda_1| > |\lambda_2| > ... > |\lambda_n|$, its inverse $A^{-1} = inv(A)$ has eigenvalues $|\mu_n| = |\frac{1}{\lambda_1}| < |\mu_{n-1}| = |\frac{1}{\lambda_2}| < ... < |\mu_1| = |\frac{1}{\lambda_n}|$. Then, to find the smallest eigenvalue of A one can compute the greatest eigenvalue of its inverse, applying the *power method for eigenvalues* to A^{-1}.
It takes as inputs: the nxn matrix A of which we compute the smallest eigenvalue, the n-dimensional starting vector of the sequence that approximates the eigenvector $v^{(0)}$, the maximum number of iterations before stopping the method maxIter, the relative tolerance of the stopping criterion relTol
The algorithm is the following:
1. while $k \leq maxIter$ and $|\mu^{(k)} - \mu^{(k-1)}| > |\mu^{(k)}|relTol$:
2. $\tilde{v}^{(k+1)} = A^{-1}v^{(k)}$
3. $\mu^{(k+1)} = v^{(k)}\tilde{v}^{(k+1)}$
4. $v^{(k+1)} = \tilde{v}^{(k+1)}/||\tilde{v}^{(k+1)}||_2^2$
5. $k = k + 1$
6. end while
7. $\lambda_n = \frac{1}{\mu^{(k+1)}}$, $v_n = v^{(k+1)}$
It outputs: the smallest eigenvalue λ_n, its corresponding n-dimensional eigenvector v_n

12.2 *deflation_method*

When for a matrix A $\in \mathbf{R^{nxn}}$ the first eigenpair (λ_1, v_1) has already been computed, *deflation method* allows to simplify the task of computing the successive greatest eigenvalue λ_2. In particular, deflation method allows to construct a matrix $P_1 \in \mathbf{R^{nxn}}$ st

$$P_1 A P_1^T = \begin{bmatrix} \lambda_1 & -b_1^T- \\ 0 & A_2 \end{bmatrix}$$

where $0 \in \mathbf{R^{n-1}}$ and $A_2 \in \mathbf{R^{(n-1)x(n-1)}}$ is a matrix having eigenvalues $|\lambda_2| > ... > |\lambda_n|$. The matrix P_1 is defined as

$$P_1 = I_n - 2\frac{(v_1 + e_1^{(n)})(v_1 + e_1^{(n)})^T}{||v_1 + e_1^{(n)}||_2^2}$$

where $e_1^{(n)} \in \mathbf{R^n}$ st $e_1^{(n)} = (1, 0, ..., 0)^T$.
It takes as inputs: the nxn matrix A, the first eigenvector v_1.

The algorithm is the following:
1. define $e_1^{(n)}$ and compute $v_1 + e_1^{(n)}$
2. compute P_1
3. compute $P_1 A P_1^T$
4. get $A_2 = P_1 A P_1^T(2:end, 2:end)$
5. get $b_1 = P_1(1, 2:end)$
It outputs:P_1, A_2, b_1, $P_1 A P_1^T$
Then, one can compute the second greatest eigenvalue of A computing the greatest eigenvalue of A_2, λ_2.
The corresponding eigenvector can also be computed: given $v_2^{(n-1)} \in \mathbf{R^{n-1}}$, eigenvector of A_2 corresponding to the eigenvalue λ_2, to find v_2 st $Av_2 = \lambda_2 v_2$, we need to "fill" the first component. To do so, we define $\bar{v_2} = (\alpha_1, v_2^{(n-1)})^T$ and compute $\alpha_1 = -\frac{b_1^T v_2^{(n-1)}}{\lambda_1 - \lambda_2}$. Then $v_2 = P_1 \bar{v_2}$.
To obtain the second smallest eigenvalue of A, we apply this procedure to A^{-1} and eigenpair (λ_n, v_n), to get a (n-1)x(n-1) matrix having eigenvalues $|\mu_2| = |\frac{1}{\lambda_{n-1}}| > |\mu_3| = |\frac{1}{\lambda_{n-2}}| > ... > |\mu_n| = |\frac{1}{\lambda_1}|$, to which we can therefore apply *inverse power method* to find λ_{n-1}, second smallest eigenvalue of A and its corresponding eigenvector v_{n-1}.

12.3 *compute_m_smallest_eigenvalues*

The function computes the m smallest eigenvalues of the nxn matrix A alternating between *inverse power method* (IPM) and *deflation method*. In particular, starting from the original matrix A, it returns *eigenvalues_A*, a m dimensional vector containing the m smallest eigenvalues of A (in increasing order), and *eigenvectors_A*, a nxm matrix having as columns the eigenvectors corresponding to the m smallest eigenvalues of A.
Since the computation of the eigenvectors is the part of the algorithm that requires the most computational cost (as it requires ...), the function also takes as input a string $both \in \{yes, no\}$ which specifies whether to also compute the eigenvectors corresponding to the m smallest eigenvalues or not. Indeed, when selecting the suitable number of clusters by evaluating the magnitude of the smallest eigenvalues (Section 5), the computation of the eigenvectors was not necessary and some computational time was saved by avoiding their computation.

The algorithm is the following:

1) computes the smallest eigenvalue (and corresponding eigenvector) of A using IPM:

$$[eigenvalues_A(1), eigenvectors_A(:,1)] = inverse_power_method(A, v0, maxIter, relTol)$$

2) iteratively uses *deflation method* to create a smaller matrix $small_Ai \in \mathrm{R}^{(\mathrm{n-(i-1)xn-(i-1)})}$ whose smallest eigenvalue is the ith smallest eigenvalue of A:
for i=2,...,m:

$$[\bar{P}_i, small_Ai, b_i, \ ] = deflation_method(small_Ai_last, v_last_smallAi)$$

3) applies IPM to find the smallest eigenvalue (and corr eigenvector) of $small_Ai$ (the eigenvalue is also the ith smaller of A):

$$[eigenvalues_A(i), v_last_smallAi] = inverse_power_method(small_Ai, v_0, maxIter, relTol)$$

4) if $both == yes$: retrieves the eigenvector of A associated to the just found ith eigenvalue;
For step 4), the procedure explained in *deflation method* has been generalized in the following way. Suppose to be at iteration i and have just found the eigenpair $(\lambda_i, v_i^{(n-(i-1))})$ of $small_Ai$.
To obtain v_i eigenvector of A the algorithm is:
1. define $\bar{v_i} = (0, ..., 0, v_i^{(n-(i-1))})$
2. for j = 1,...,i-1:
3. compute the current last zero component of $\bar{v_i}$: $\alpha_{i-j} = -\frac{b_{i-j}^T \bar{P}_{i-(j-1)} v_i^{(n-(i-j))}}{\lambda_{i-j} - \lambda_i}$
4. update $\bar{v_i}(i-j) = \alpha_{i-j}$
5. update $v_i^{(n-(i-(j+1)))} = (\alpha_{i-j}, v_i^{(n-(i-j))})$
6. end for
7. for k=1,...,i-1:
8. $\bar{v_i} = P_{i-k} \bar{v_i}$
10. end for
11. $v_i = \bar{v_i}$

13 Conclusions

In conclusion, the self-implemented spectral clustering method, tested on the Circle and Spiral datasets, proves to be effective for clustering purposes. However, the Matlab spectral clustering method outperforms the self-implemented version due to its superior construction of the similarity matrix, of the graph Laplacian, and computation of eigenvalues, resulting in more accurate clustering outcomes.
The failure of k-means and k-medoids methods can be explained by the fact that they fail to capture the structure of the datasets, as they rely on centroid-based distance measures which are not suitable in this case.
Moreover, the reduced performance of the method as the parameter k increases is probably due to the fact that points may become nearest neighbors with others that are not directly adjacent on the spirals or circles, but are still sufficiently close to avoid being discarded. The optimal value is $k = 10$, where the clusters produced by the self-implemented spectral clustering method match those obtained with Matlab's spectral clustering method.
The method also demonstrates notable efficiency, even when handling datasets of significant size and the associated computational demands of constructing and managing large matrices. It would be interesting to apply the self-implemented spectral clustering method to other datasets to further evaluate its reliability and performance.
Finally, the use of the normalized Laplacian matrix does not improve the results; instead, it seems to behave similarly to centroid-based or radius-based methods. This behavior is likely due to the scaling of L of $D^{-1/2}$, which balances the contributions of nodes based on their

degrees, thereby treating all nodes in a more uniform manner, which does not seem suitable in this case.